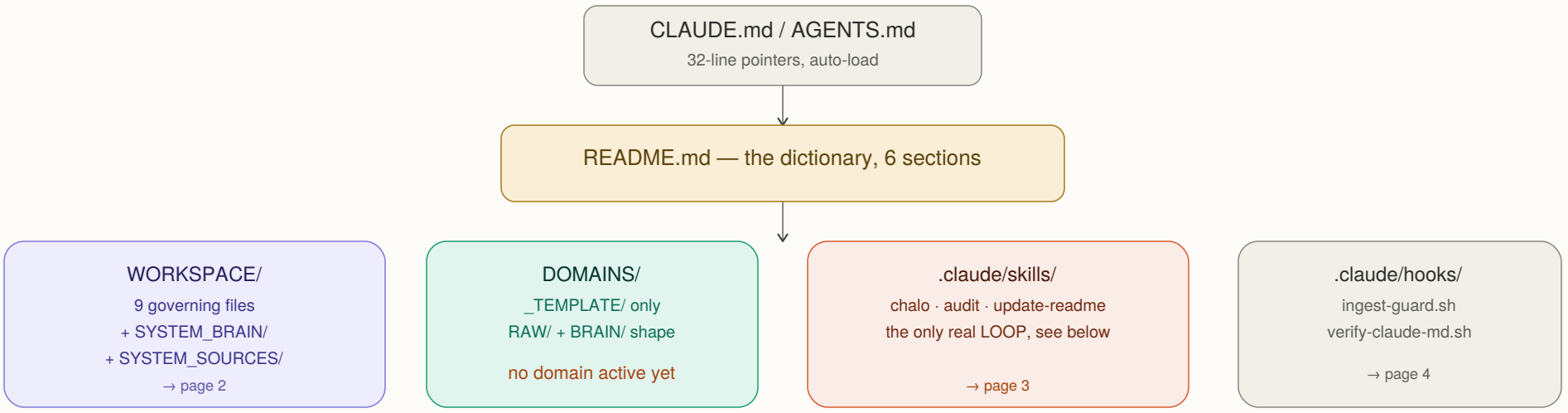
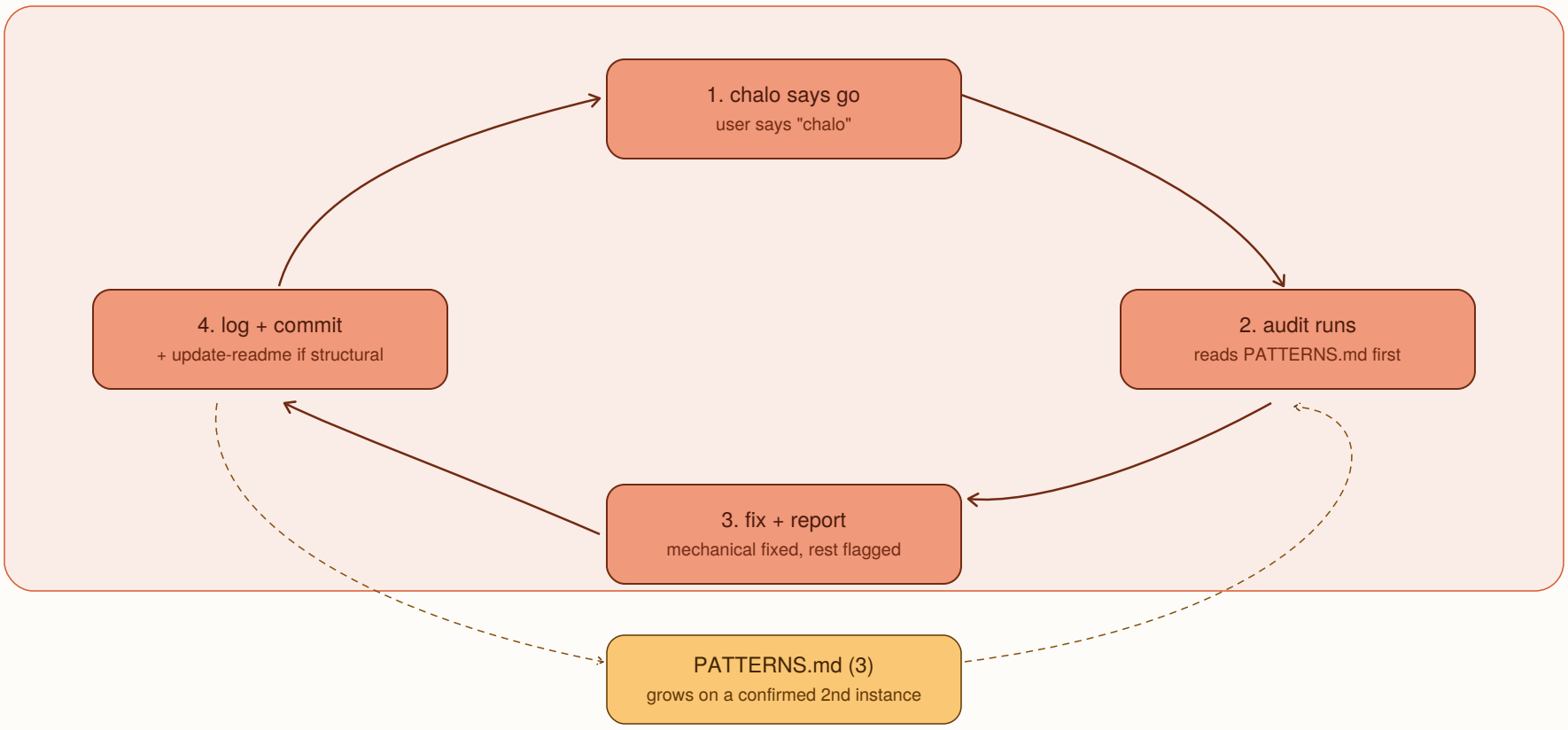
SYSTEM_DESIGN_OS — Eagle's-Eye Overview



The one real loop — triggered by "chalo," repeats every session close



README.md is rebuilt, not the loop's source — see page 5 for the DNA and the honest limit of "self-learning."

WORKSPACE — governance + memory

domains — none active

skills — the loop

hooks — mechanical, no skipping

Built and tested entirely on itself. The single open item: bring a real domain.

WORKSPACE/ — Governance & Memory

Page 2 of 5. README.md sections 3.1 and 3.2. 9 files + 2 memory folders.

OPERATING_CONTRACT.md

Division of labor + 9 behavioral rules: verify before claiming, one issue at a time, surface problems, push back, the sandbox isn't real, don't assume the topic, log without being asked, every CC-prompt gets an ID, document tool-specific work's portable concept too.

INGEST.md

Verification protocol for any source.
0 route → 1 confirm readable → 2 read real content → 3 produce checkable proof → 4 self-check → 5 write the record (INGESTED/PARTIAL/FAILED) → 6 reconcile into the brain (extends/conflicts/new).
Core rule: no claim beyond the evidence.

FRAMEWORK.md

Why, not just what. Opens with the DNA statement (page 5). Three pillars: LLM-Wiki knowledge accumulation, ICM-style numbered workflow folders (+10 each), markdown as the only artifact format.
Plus mechanism-agnostic specs for every .claude/-specific thing (page 4).

RULES.md

Expansion patterns, written only once proven. Holds: how to copy and start a real domain from _TEMPLATE/, the +10 stage-numbering rule, and a concrete file-count threshold (~50-100/brain) for when Obsidian/vector search is worth it.

PATTERNS.md — 3 confirmed

P001 known principle not self-applied
P002 clean pass not independently checked
P003 correct prompt never reaches the tool

Each needs 2 confirmed instances to earn an entry. audit reads this file FIRST, every run, before its own 5 checks.

DECISIONS.md / REASONING.md

DECISIONS — the few choices formally LOCKED: what, why, what triggered it, what would reopen it. Most changes live in EVOLUTION_LOG instead.

REASONING — the long version behind a LOCKED decision: rejected alternative + what would change it.

EVOLUTION_LOG.md

Full chronological history. Every real change logged in the same pass it happened — mistakes left visible, never edited away after the fact.

STATUS.md

Current snapshot — distinct from the log. What's verified, what's open, what to check first. Refreshed by chalo at session close, not after every change.

SYSTEM_BRAIN/ + SYSTEM_SOURCES/

BRAIN: sources/ + concepts/ + context/ (overview.md = from docs, conversational.md = from dialogue, judgment-promoted).
SOURCES: the 4 raw docs, immutable.

DOMAINS/_TEMPLATE/ — the reusable pattern, identical shape to SYSTEM_BRAIN/ above

Copy this folder, rename it, drop real sources into its RAW/, run them through INGEST.md.
RAW/ and BRAIN/ are never shared across domains — each domain's knowledge stays isolated.

No domain currently exists. This is the single open item since session one.

.claude/skills/ — The Three Skills, In Full

chalo — end-of-session closing ritual

Trigger: saying "chalo" or typing /chalo when ending a session.

0 Run audit first — unconditionally, as its own step zero

1 Review what actually happened against EVOLUTION_LOG.md

2 Confirm the log is current, backfilling honestly if missed

3-4 Write dialogue-derived insight to conversational.md;
check if it's grown enough to promote into overview.md

5a Rewrite STATUS.md to match reality, not just append

5b Flag anything stated as fact but never actually checked

5c Invoke update-readme IF a domain/skill/hook/file changed

6 Commit everything in one final pass, report back honestly

audit — system-wide self-check, distinct from a single session

Trigger: automatic, as chalo's step 0, every session close.

0 Read PATTERNS.md first — apply a matching countermeasure, don't rediscover

1 Does every file actually do what its own stated purpose says?
(uses git history as evidence, not assumption)

2 Has a judgment-based promotion condition quietly been crossed?

3 Has a known principle been applied once, but skipped elsewhere it applies?

4 Broken references AND undocumented real structure — both directions

5 Rules in OPERATING_CONTRACT.md with no EVOLUTION_LOG.md entry ever exercising them

6 Tag every finding MECHANICAL (fix + always report) or JUDGMENT-REQUIRED (surface, never auto-resolve)

6b If this follows a self-correction, actively try to disprove the clean result before reporting it
— added after a clean pass was wrong, twice, in the same session

7 Log the audit itself, with findings or an explicit "nothing found"

Known limitation: catches a SECOND instance of a named pattern. Cannot invent a brand-new one on its own.

update-readme — keeps the dictionary current

Trigger: chalo's step 5c (structural change) or explicit request — never standalone.

Fires on: new domain, new skill, new hook, new WORKSPACE file, or any rename/removal.

Reads the real content first (never the name alone) → finds the correct section →
writes the entry matching its neighbors' style → checks for duplication → commits → logs.

Distinguishes the architecture tree (compressed shorthand) from full dictionary entries
(section 3 vs. 3.1/4/5) — added after a real test found this judgment was unstated.

Never fires for DOMAINS/_TEMPLATE/ itself — it's a template, not a real, active thing yet.

.claude/hooks/ — Mechanical, Cannot Be Talked Past

A rule in markdown can be skipped under pressure to seem helpful. A hook is a script that runs regardless of what Claude decides in the moment — that is the entire point.

ingest-guard.sh

Fires: PreToolUse, before any file write completes.

Blocks any write into:

DOMAINS/ · SYSTEM_BRAIN/ · SYSTEM_SOURCES/

Unless the path is inside a _TEMPLATE/ folder.

Why a hook and not a rule: a written instruction to "route correctly first" can be skipped if proceeding feels more helpful than asking. This cannot.

Built after a real near-miss: domain content sharing the same RAW/BRAIN/ as the system itself.

verify-claude-md.sh

Fires: SessionStart, at the beginning of every session.

Confirms the CLAUDE.md that loaded is the genuine one for THIS project — not a stale copy, not a different project's file.

Uses the official CLAUDE_PROJECT_DIR variable Claude Code itself sets — not a hand-written guess at the project root.

An earlier version manually walked up the directory tree and was replaced once the official variable was found to do the same job more reliably.

Silent on success — you only see output if something is actually wrong.

Known, permanent limitation of both hooks

They are Claude-Code-specific. A different agent (Codex, or otherwise) would need its own equivalent in its own mechanism — FRAMEWORK.md's mechanism-agnostic specs exist precisely so that rebuild has a real blueprint, rather than starting from zero.

The DNA — Five Qualities, Each Tied to a Real Mechanism

Agent-agnostic

CLAUDE.md + AGENTS.md identical & synced. Tool-specific code kept separate from its portable concept in FRAMEWORK.md.

Transferable

No database, no proprietary format. Copyable wholesale to another person, agent, or future session — no translation step.

Scalable

Knowledge accumulates per-domain, never shared. A concrete, sourced threshold (RULES.md) for when index files stop being enough.

Self-auditing

audit checks the system against its own stated rules, automatically, every session via chalo — not dependent on anyone remembering.

Ever-learning

PATTERNS.md converts a confirmed second instance into a standing countermeasure, checked before a third instance recurs.

The honest limit, stated once and meant to be remembered

This system gets better at not repeating a KNOWN mistake. It does not recognize a brand-new pattern from a single occurrence — that still requires a human noticing, the same way every pattern in PATTERNS.md was found by the user, not the system.

"Self-learning" here means compounding known lessons reliably — not discovering unknown ones.

What this system is not, yet

There is no business logic here. No domain has been ingested, no workflow stage has been built, no revenue model has been tested. Every mechanism on every page of this document has only ever been exercised on the construction of this scaffold itself.

The single open item, unchanged since the very first message: bring a real domain and run it through DOMAINS/_TEMPLATE/ for real.

← page 4 hooks · page 1 overview